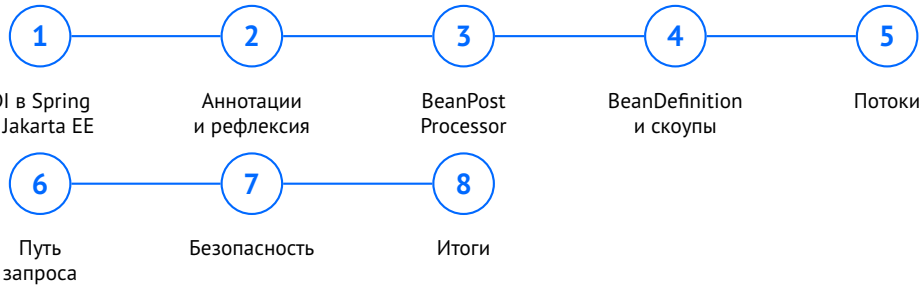
ИТМО

Spring и Jakarta EE

Рефлексия, контейнеры бинов и прокси

Санкт-Петербург, 6 октября 2026 г.

План лекции



Обе платформы стоят на **рефлексии и аннотациях**.
Spring даёт залезть в контейнер руками, Jakarta EE — стандарт и гарантии.

Эндпоинт в Spring MVC

```
@RestController
@RequestMapping("/students")
public class StudentController {
  private final StudentService service;

  public StudentController(StudentService service) {
    this.service = service;
  }

  @GetMapping("/{id}")
  public StudentDto get(@PathVariable long id) {
    return service.find(id);
  }

  @PostMapping
  @ResponseStatus(HttpStatus.CREATED)
  public StudentDto create(
      @Valid @RequestBody CreateStudentRequest request) {
    return service.create(request);
  }
}
```

- `@RestController` — класс обрабатывает HTTP, ответы сразу в JSON
- `@RequestMapping` — общий префикс пути
- `@GetMapping`, `@PostMapping` — метод и путь
- `@PathVariable` — часть пути, `@RequestBody` — тело запроса
- `@Valid` — проверка тела, при ошибке 400
- Возвращаемый объект превращается в JSON сам

Эндпоинт в Jakarta REST

```
@ApplicationPath("/api")
public class App extends Application {}

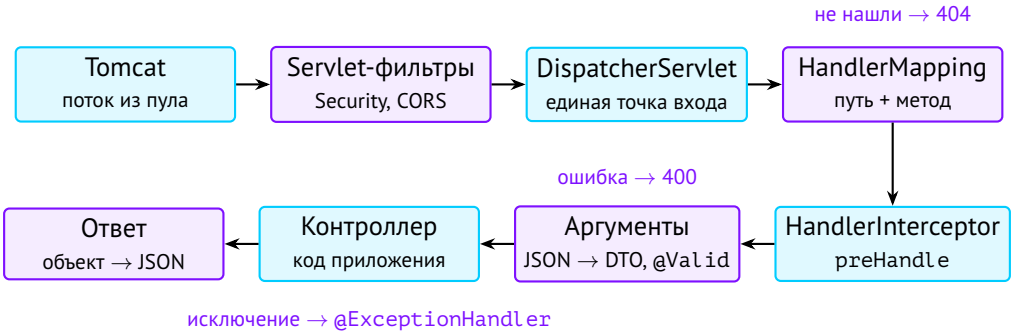
@Path("/students")
@RequestScoped
public class StudentResource {
  @Inject StudentService service;

  @GET @Path("/{id}")
  @Produces(MediaType.APPLICATION_JSON)
  public StudentDto get(@PathParam("id") long id) {
    return service.find(id);
  }

  @POST
  @Consumes(MediaType.APPLICATION_JSON)
  public Response create(
      @Valid CreateStudentRequest request) {
    StudentDto created = service.create(request);
    return Response.status(201).entity(created).build();
  }
}
```

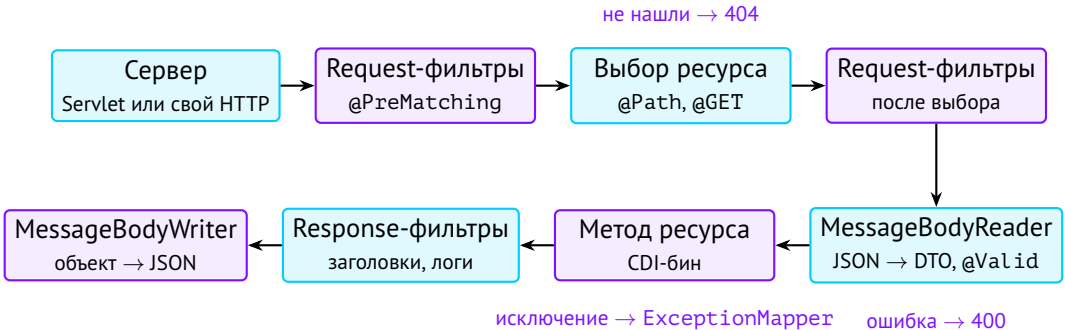
- `Application` + `@ApplicationPath` — точка входа и префикс всего API
- `@Path` — путь, `@GET`, `@POST` — метод
- `@Produces`, `@Consumes` — формат ответа и тела
- `@PathParam` — часть пути; тело — параметр **без** аннотации
- `Response` — когда нужен свой статус или заголовки

Путь запроса в Spring MVC



Tomcat знает только про сервлеты. Spring MVC — это **один** сервлет, `DispatcherServlet`, который дальше раздаёт запросы по бинам-контроллерам.

Путь запроса в Jakarta REST



Jakarta REST обычно работает поверх Servlet, но не обязан: Quarkus, например, запускает его на своём HTTP-сервере. Фильтры, reader'ы, writer'ы и `ExceptionMapper` — классы с аннотацией `@Provider`.

Внедрение зависимостей в Spring

```
@Service
public class StudentService {
  private final StudentRepository repo;
  private final Clock clock;

  @Autowired
  public StudentService(
      StudentRepository repo,
      Clock clock) {
    this.repo = repo;
    this.clock = clock;
  }
}
```

```
@Configuration
public class AppConfig {
  @Bean  // Clock is a JDK class
  Clock clock() {
    return Clock.systemUTC();
  }
}
```

- `@Autowired` — точка внедрения; при одном конструкторе можно не писать
- `@Bean` — бин из класса JDK или библиотеки: на `Clock` аннотацию не повесить

Куда внедрять: конструктор, поле, сеттер

Конструктор

```
private final Clock clock;

@Autowired
StudentService(Clock c) {
  this.clock = c;
}
```

Поле

```
@Autowired
private Clock clock;
```

Сеттер

```
private Clock clock;

@Autowired
void setClock(Clock clock) {
  this.clock = clock;
}
```

	Плюсы	Минусы
Конструктор рекомендуется	поле `final`; зависимости видны в сигнатуре; в тесте — `new`; цикл виден при старте	больше кода
Поле	короче всего	рефлексия по `private`; в тесте без контейнера не заполнить
Сеттер	зависимость можно заменить или не передать	объект может остаться без неё

Внедрение зависимостей в Jakarta EE

ИТМО

```
@ApplicationScoped
public class StudentRepository { ... }

@ApplicationScoped
public class StudentService {
  @Inject
  StudentRepository repo;
}
```

```
@ApplicationScoped
public class ClockProducer {
  @Produces
  Clock clock() {
    return Clock.systemUTC();
  }
}
```

- Бин — класс с аннотацией скоупа
- `@Inject` — на поле, конструкторе или сеттере
- `@Produces` — аналог `@Bean`
- Контейнер поднимает сервер приложений, `main` нет

Две модели запуска

Jakarta EE

Приложение — **WAR**, его разворачивают на **сервере приложений**.

Все реализации уже в сервере. DataSource, транзакции и безопасность настраиваются **в сервере**, приложение берёт их по имени (JNDI).

Spring Boot

Приложение — **fat jar**: код, библиотеки и встроенный Tomcat.

Настройки — в `application.yml`.

Запуск: `java -jar app.jar`

Servlet API — это WSGI для Java, Tomcat — аналог gunicorn

Как это работает?

```
@Service
public class StudentService {
  @Transactional
  public void enroll(long id) { ... }
}

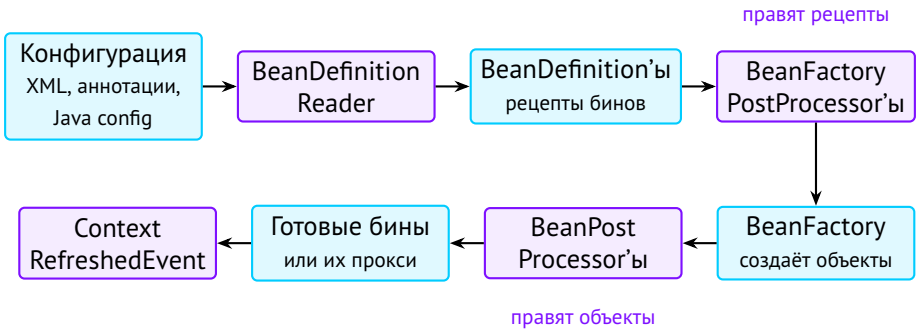
StudentService s =
    context.getBean(StudentService.class);
System.out.println(s.getClass().getName());

> StudentService$$SpringCGLIB$$0
```

Вопросы

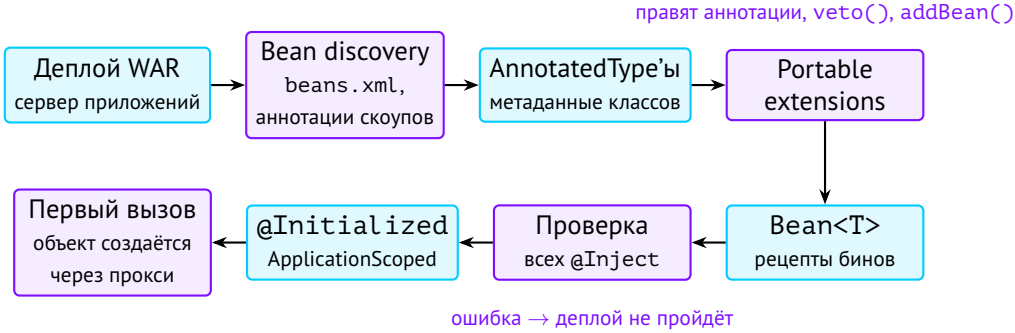
- Кто создал объекты и связал их между собой?
- Откуда контейнер узнал про `@Service`?
- Как значение попало в `private`-поле?
- Почему класс бина — не `StudentService`?

Как поднимается контекст Spring



`BeanFactory` — базовый контейнер. `ApplicationContext` расширяет его: сам находит процессоры, создаёт singleton'ы при старте, публикует события.

Как поднимается контейнер CDI



Spring создаёт singleton'ы при старте. CDI создаёт объекты нормальных скоупов **лениво** — при первом вызове через клиентский прокси.

Аннотация ≠ декоратор

Python: декоратор

```
@lru_cache      # wraps it
def get(isu): ...
```

Декоратор — функция. Он **сам** оборачивает `get` в момент объявления.

Java: аннотация

```
@Deprecated  // just a label
public void get(int isu) {...}
```

Аннотация — **пассивная метка**. Сама по себе ничего не делает.

Объявление аннотации

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface InjectRandomInt {
  int min();
  int max();
}

// usage
@InjectRandomInt(min = 2, max = 7)
private int repeat;
```

- `@interface` — объявление аннотации, параметры похожи на методы
- `@Target` — куда можно повесить: класс, поле, метод, параметр
- `@Retention` — сколько аннотация живёт:

  `SOURCE` — только в исходнике

  `CLASS` — в байткоде, в рантайме не видна

  `RUNTIME` — видна через рефлексию

Без `RUNTIME` фреймворк аннотацию не увидит

Рефлексия: класс как объект

```
Class<?> c = obj.getClass();

for (Field f : c.getDeclaredFields()) {
  InjectRandomInt a = f.getAnnotation(
      InjectRandomInt.class);
  if (a != null) {
    f.setAccessible(true);  // open private
    f.setInt(obj, a.min());
  }
}

c.getMethod("sayQuote").invoke(obj);
```

- `Class<?>` — описание класса во время работы программы
- `getDeclaredFields()` — все поля, включая `private`
- `getAnnotation(...)` — прочитать метку и её параметры
- `setAccessible(true)` — обойти `private`
- `invoke` — вызвать метод по имени

Сквозной пример

```
public interface Quoter {
  void sayQuote();
}

public class TerminatorQuoter
    implements Quoter {
  private int repeat;
  private String message = "I'll be back";

  public void sayQuote() {
    for (int i = 0; i < repeat; i++)
      System.out.println(message);
  }
}
```

- Цитатник терминатора: печатает фразу `repeat` раз
- Далее на класс навешиваются собственные аннотации
- Для каждой — своя инфраструктура, которая её обрабатывает
- Для каждой задачи — решение в Jakarta EE

Задача: @InjectRandomInt

```
public class TerminatorQuoter
    implements Quoter {

  @InjectRandomInt(min = 2, max = 7)
  private int repeat;

  private String message = "I'll be back";
  ...
}
```

Вопрос

Требуется: фраза повторяется случайное число раз, от 2 до 7.

Аннотация — просто метка. **Кто положит значение в поле?**

Первый BeanPostProcessor

```
public class InjectRandomIntAnnotationBeanPostProcessor
    implements BeanPostProcessor {

  @Override
  public Object postProcessBeforeInitialization(Object bean, String name) {
    for (Field f : bean.getClass().getDeclaredFields()) {
      InjectRandomInt a = f.getAnnotation(InjectRandomInt.class);
      if (a != null) {
        int v = ThreadLocalRandom.current().nextInt(a.min(), a.max() + 1);
        f.setAccessible(true);
        ReflectionUtils.setField(f, bean, v);
      }
    }
    return bean;
  }
}
```

Это та же рефлексия, только её запускает контейнер для **каждого** бина

ИТМО

BeanPostProcessor	Что обрабатывает
`AutowiredAnnotationBeanPostProcessor`	`@Autowired`, `@Value`
`CommonAnnotationBeanPostProcessor`	`@PostConstruct`, `@PreDestroy`
`AnnotationAwareAspectJAutoProxyCreator`	`@Transactional`, `@Async`



«Магия» Spring — это набор BeanPostProcessor'ов и рефлексия.
Свой BPP — такой же полноправный участник, как встроенные.

Jakarta EE: producer вместо BPP

```
@Qualifier @Retention(RUNTIME)
public @interface RandomInt {
  @Nonbinding int min() default 0;
  @Nonbinding int max() default 10;
}

@ApplicationScoped
public class RandomProducer {
  @Produces @RandomInt
  int random(InjectionPoint ip) {
    RandomInt a = ip.getAnnotated()
        .getAnnotation(RandomInt.class);
    return ThreadLocalRandom.current()
        .nextInt(a.min(), a.max() + 1);
  }
}
```

Использование:

```
@Inject @RandomInt(min = 2, max = 7) int repeat;
```

- `@Produces` — метод, который производит значение для внедрения
- `InjectionPoint` — куда внедряем: можно прочитать аннотацию поля
- `@Nonbinding` — параметры не влияют на выбор producer'а

Spring даёт залезть в любой объект. CDI даёт типобезопасную точку расширения.

Динамический прокси

```
Quoter real = new TerminatorQuoter();

Quoter proxy = (Quoter) Proxy.newProxyInstance(
    Quoter.class.getClassLoader(),
    new Class<?>[]{ Quoter.class },
    (p, method, args) -> {
      System.out.println("before");
      return method.invoke(real, args);
    });

proxy.sayQuote();
```

- JVM **на лету** создаёт класс, который реализует `Quoter`
- Каждый вызов попадает в лямбду — можно сделать что-то до и после
- Работает **только через интерфейс**
- Нет интерфейса — CGLIB или ByteBuddy генерируют **подкласс**, поэтому `final` обернуть нельзя

Так BeanPostProcessor может подменить бин обёрткой — следующий слайд

@Profiling: свой прокси

```
// before: remember the original class
public Object postProcessBeforeInitialization(
    Object bean, String name) {
  Class<?> c = bean.getClass();
  if (c.isAnnotationPresent(Profiling.class))
    map.put(name, c);
  return bean;
}
```

`map` — поле BPP: имя бина → его исходный класс

```
// after: return a proxy
public Object postProcessAfterInitialization(
    Object bean, String name) {
  Class<?> c = map.get(name);
  if (c == null) return bean;
  return Proxy.newProxyInstance(
      c.getClassLoader(), c.getInterfaces(),
      (proxy, m, args) -> {
        long t = System.nanoTime();
        try { return m.invoke(bean, args); }
        finally { log(m, System.nanoTime() - t); }
      });
}
```

Почему именно так

Зачем `map`

К фазе `after` бин может быть уже обёрнут **другим** BPP, и аннотацию на прокси не найти. Поэтому класс запоминается в `before`, пока объект ещё настоящий.

Почему прокси в `after`

Если вернуть прокси в `before`, следующие BPP и `@PostConstruct` получат прокси вместо настоящего объекта и не найдут его поля.

- JDK `Proxy` требует интерфейс, иначе — CGLIB-подкласс (по умолчанию в Spring Boot)
- Профилирование можно включать и выключать на лету, например через JMX
- **Ответ на вопрос из начала:** `@Transactional` обрабатывает такой же BPP (`AnnotationAwareAspectJAutoProxyCreator`). Отсюда `$$SpringCGLIB$$0`

Jakarta EE: перехватчик

```
@InterceptorBinding
@Retention(RUNTIME) @Target({TYPE, METHOD})
public @interface Profiling {}

@Profiling @Interceptor
@Priority(Interceptor.Priority.APPLICATION)
public class ProfilingInterceptor {

  @AroundInvoke
  Object time(InvocationContext ctx) throws Exception {
    long t = System.nanoTime();
    try {
      return ctx.proceed();   // the real method
    } finally {
      log(ctx.getMethod(), System.nanoTime() - t);
    }
  }
}

@Profiling
@ApplicationScoped
public class TerminatorQuoter implements Quoter { ... }
```

- `@InterceptorBinding` — аннотация-связка: где стоит `@Profiling`, там работает перехватчик
- `@AroundInvoke` — код вокруг вызова, `ctx.proceed()` вызывает настоящий метод
- Обёртку-подкласс контейнер создаёт сам — свой BPP и `Proxy` не нужны
- `@Priority` включает перехватчик; без него он молча выключен

Трёхфазный конструктор

Вопрос залу: почему в конструкторе `repeat == 0`?

Фаза	Что уже есть	Чего ещё нет
1. Конструктор	только сам объект	зависимостей и полей от BPP
2. `@PostConstruct`	зависимости, поля от BPP	**прокси**: `@Transactional` и профилирование здесь не работают
3. После старта контекста	всё, включая прокси	—

`@PostConstruct` вызывает BPP в `before`, а прокси появляется позже — в `after`

Фаза 3: зачем она нужна

К третьей фазе **все BPP уже отработали**: бин обёрнут прокси, `@Transactional`, `@Async`, профилирование работают

Spring

```
@Component
public class Warmup {
  @Autowired StudentService service; // proxy

  @EventListener(ContextRefreshedEvent.class)
  public void warmUp() {
    service.loadDictionaries();   // tx works
  }
}
```

Jakarta EE

```
@ApplicationScoped
public class Warmup {
  @Inject StudentService service;

  void init(@Observes
      @Initialized(ApplicationScoped.class)
      Object e) {
    service.loadDictionaries();   // tx works
  }
}
```

В `@PostConstruct` тот же вызов прошёл бы **без транзакции**: прокси ещё не создан.

BeanDefinition — рецепт бина

Поле	Откуда берётся
`beanClassName`	класс с `@Component`
`scope`	`@Scope`
`lazyInit`	`@Lazy`
`primary`	`@Primary`
`factoryBeanName`, `factoryMethodName`	метод `@Bean`
`initMethodName`, `destroyMethodName`	`@Bean(initMethod)`

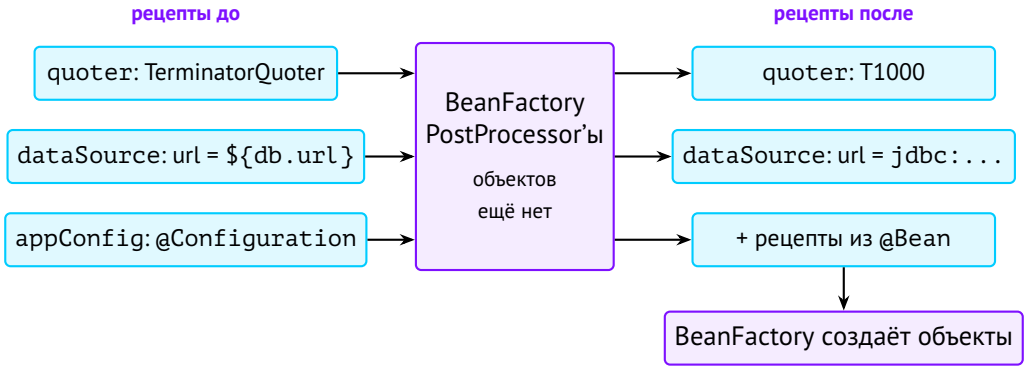
Рецепт и блюдо

Контейнер стартует в две фазы: сначала **описания** всех бинов, потом **объекты**.

Аналог в CDI — `Bean<T>`.

У бина из `@Bean` `beanClassName == null`: рецепт — «вызови метод»

Как работает BeanFactoryPostProcessor



свой BFPP по аннотации `@DeprecatedClass`

`PropertySourcesPlaceholderConfigurer`

`ConfigurationClassPostProcessor`

меняет класс в рецепте: создастся `T1000`

подставляет значения `${...}` из настроек

разбирает `@Configuration` и добавляет новые рецепты

BFPP и BPP

	BeanFactoryPostProcessor	BeanPostProcessor
Работает с	рецептами	объектами
Когда	до создания любых бинов	вокруг инициализации каждого бина
Пример	`ConfigurationClassPostProcessor` разбирает `@Configuration` и `@Bean`	`AutowiredAnnotationBeanPostProcessor` внедряет `@Autowired`, `@Value`
Пример	`PropertySourcesPlaceholderConfigurer` подставляет `${...}`	`AnnotationAwareAspectJAutoProxyCreator` создаёт прокси для `@Transactional`

- Источник рецептов любой: XML, аннотации, Java config, даже `.properties`. `@Configuration` проксируется CGLIB
- Ошибки старта — ошибки рецептов: `NoSuchBeanDefinitionException`, `NoUniqueBeanDefinitionException`

Jakarta EE: рецепты в CDI

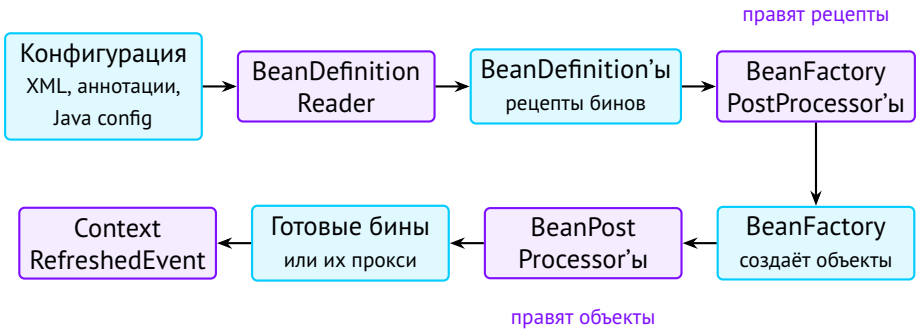
Spring	CDI
`@Bean`	`@Produces`
`@Primary`	`@Alternative` + `@Priority`
BFPP	`ProcessAnnotatedType`: поменять аннотации, `veto()`
регистрация вручную	`AfterBeanDiscovery .addBean()`
поиск по имени и типу	поиск по **типу и квалификатору**

Разница подходов

В Spring `BeanDefinition` — изменяемые данные, BFPP правит в них что угодно.

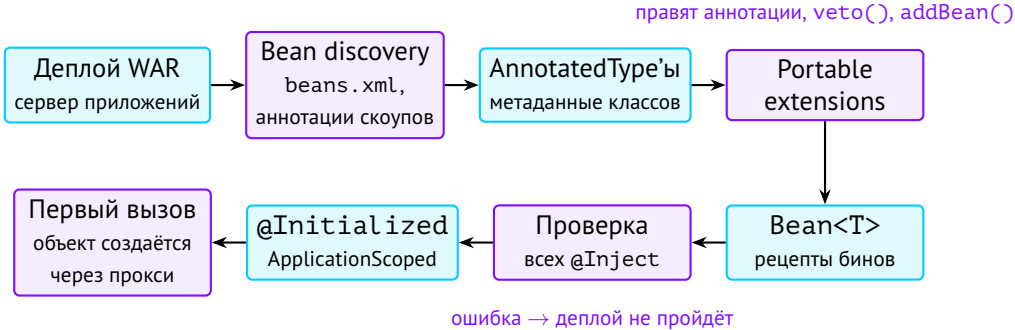
В CDI `Bean<T>` строится из аннотаций: меняют **аннотации класса**, а не рецепт.

Как поднимается контекст Spring



`BeanFactory` — базовый контейнер. `ApplicationContext` расширяет его: сам находит процессоры, создаёт singleton'ы при старте, публикует события.

Как поднимается контейнер CDI



Spring создаёт singleton'ы при старте. CDI создаёт объекты нормальных скоупов **лениво** — при первом вызове через клиентский прокси.

Spring и CDI: соответствие этапов

Spring	CDI
`main` → `ApplicationContext`	деплой на сервер, сервер поднимает контейнер
reader → `BeanDefinition`	bean discovery → `AnnotatedType` → `Bean<T>`
`BeanFactoryPostProcessor`	portable extensions: события `ProcessAnnotatedType`, `AfterBeanDiscovery`
`BeanPostProcessor` → объекты	`Bean<T>.create()` + перехватчики
`ContextRefreshedEvent`	событие `@Initialized(ApplicationScoped.class)`
ошибка зависимости — при создании бина	**все точки внедрения проверяются при деплое**

Скоупы

Spring	CDI	Python (FastAPI)
singleton (по умолчанию)	`@ApplicationScoped`	`app.state`
request	`@RequestScoped`	`Depends` с `yield`
prototype	`@Dependent` (по умолчанию)	—
session	`@SessionScoped`	—

Singleton-скоуп $\neq$ паттерн Singleton:

один экземпляр на контейнер, без `getInstance()`

Пример со скринсейвером

```
@Component       // singleton
public abstract class ColorFrame
    extends JFrame {

  @Lookup        // prototype
  protected abstract Color getColor();

  public void showOnRandomPlace() {
    getContentPane()
        .setBackground(getColor());
  }
}
```

Обычный `@Autowired Color` внедрится **один раз** — окно всегда одного цвета.

Spring:

- `@Lookup` — Spring генерирует подкласс, который берёт бин из контекста
- `ObjectProvider<Color>`
- `proxyMode` — снова прокси

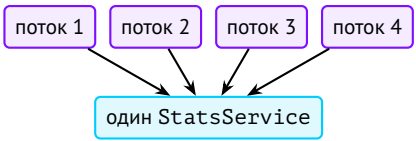
Jakarta EE

Нормальные скоупы **всегда** внедряются через клиентский прокси — проблемы нет. Для `@Dependent`: `Instance<Color>` и `get()`.

Singleton и потоки

```
@Service
public class StatsService {
  private int requests;  // shared

  public void hit() {
    requests++;          // race!
  }
}
```



- Tomcat обрабатывает каждый запрос в своём потоке, пул по умолчанию — 200
- `requests++` — «прочитать — изменить — записать»: GIL в Java нет, потоки работают параллельно

В Spring

Потокобезопасность целиком на вас. Не храните состояние в singleton, а если нужно — `AtomicInteger`, `ConcurrentHashMap`.

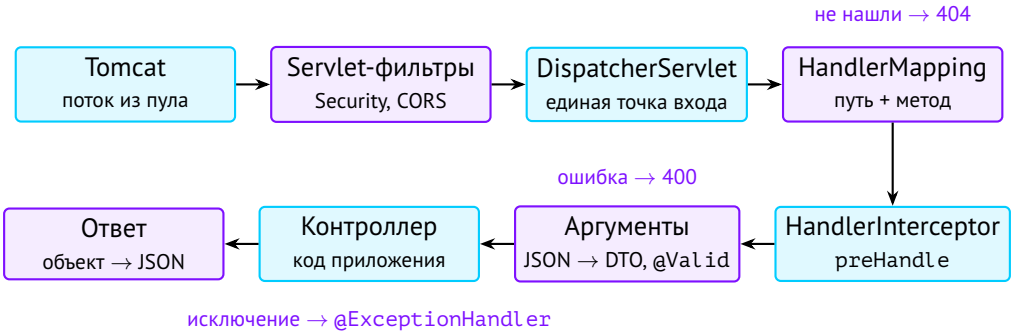
EJB: потоки берёт на себя контейнер

EJB	Как работает
`@Stateless`	**пул экземпляров**: каждый обслуживает один вызов за раз, гонок нет
`@Singleton`	один экземпляр, по умолчанию `@Lock(WRITE)`: потоки входят по одному. `@Lock(READ)` — параллельно
`@Stateful`	экземпляр на клиента, похоже на session-скоуп
`@Asynchronous`	асинхронный вызов — как `@Async` в Spring
`@Schedule`	задача по расписанию — как `@Scheduled` в Spring

Jakarta EE защищает разработчика ценой гибкости и скорости.

Spring доверяет разработчику

Путь запроса в Spring MVC



Tomcat знает только про сервлеты. Spring MVC — это **один** сервлет, `DispatcherServlet`, который дальше раздаёт запросы по бинам-контроллерам.

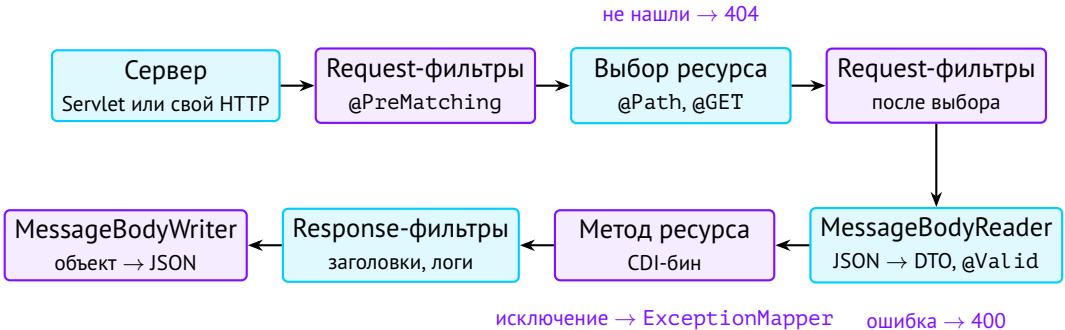
Кто что делает внутри DispatcherServlet

ИТМО

Компонент	Роль
`HandlerMapping`	найти метод контроллера по пути и HTTP-методу (`@GetMapping`)
`HandlerAdapter`	вызвать найденный метод
`HandlerMethodArgumentResolver`	собрать параметры: `@PathVariable`, `@RequestParam`, `@RequestBody`
`HttpMessageConverter`	JSON ↔ объект (Jackson)
`HandlerInterceptor`	код до и после контроллера, уже знает выбранный метод
`HandlerExceptionResolver`	исключение → ответ: `@ExceptionHandler`, `@RestControllerAdvice`

Все компоненты — обычные бины контекста: любой можно заменить или добавить свой

Путь запроса в Jakarta REST

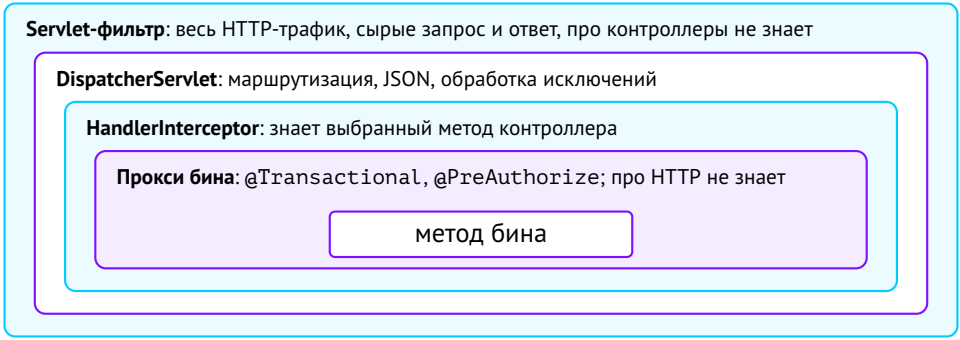


Jakarta REST обычно работает поверх Servlet, но не обязан: Quarkus, например, запускает его на своём HTTP-сервере. Фильтры, reader'ы, writer'ы и `ExceptionMapper` — классы с аннотацией `@Provider`.

Spring MVC и Jakarta REST

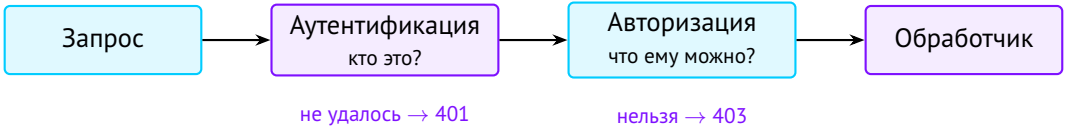
	Spring MVC	Jakarta REST
Точка входа	`DispatcherServlet`	`Application` + `@ApplicationPath`
Маршрут	`@GetMapping("/{id}")`	`@Path("/{id}")` + `@GET`
Параметры	`@PathVariable`, `@RequestParam`, `@RequestBody`	`@PathParam`, `@QueryParam`, тело без аннотации
JSON	`HttpMessageConverter` (Jackson)	`MessageBodyReader/Writer` (JSON-B)
До и после	`Filter`, `HandlerInterceptor`	`ContainerRequestFilter`, `ContainerResponseFilter`
Ошибки	`@ExceptionHandler`	`ExceptionMapper`
Жизнь контроллера	singleton на всё приложение	по умолчанию **новый экземпляр на запрос**

Где срабатывает какая обёртка



В Jakarta REST: `ContainerRequestFilter` — уровень фильтров и interceptor'ов, CDI-перехватчик — уровень прокси бина

Аутентификация и авторизация



Аутентификация

Проверка личности: логин и пароль, токен, сертификат.

Результат — известный пользователь с набором ролей.

Авторизация

Проверка прав: роль ADMIN может удалять, USER — только читать.

Без аутентификации невозможна: сначала «кто», потом «что можно».

Как устроен JWT

header
`{alg: RS256}`

payload
`{sub: ivanov, roles: [USER], exp: 1767225600}`

signature
подпись header + payload

каждая часть в base64url, между ними точки

Открыт

Токен не зашифрован: payload читает кто угодно — секреты туда не кладут. Подпись гарантирует, что токен **не подделан**.

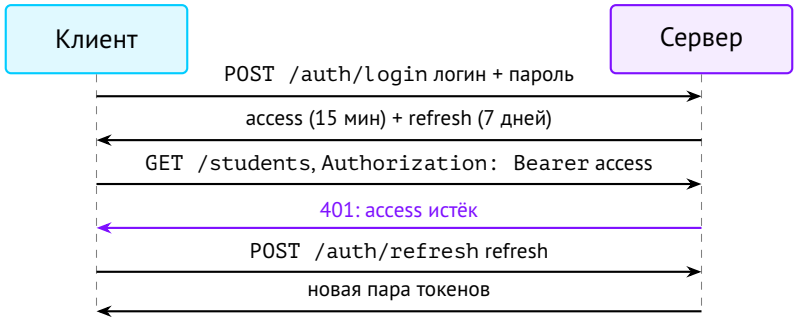
Подпись

HS256 — общий секрет.
RS256 — подпись приватным ключом, проверка публичным.

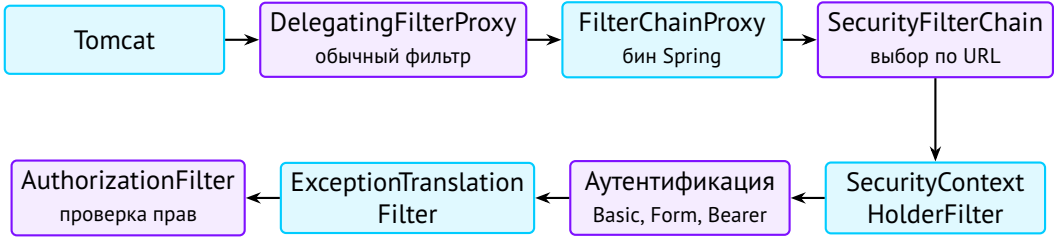
Без сессии

Сервер ничего не хранит: проверяет подпись и exp. Минус — выданный токен не отозвать до истечения.

Access- и refresh-токены

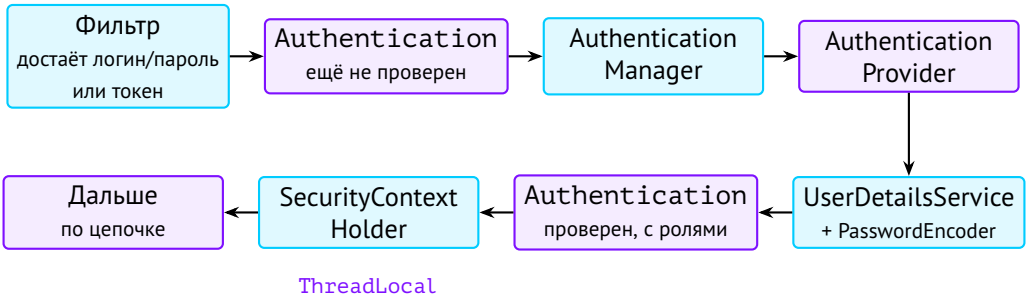


Spring Security: цепочка фильтров



Spring Security целиком живёт в **servlet-фильтрах**, до Spring MVC. `DelegatingFilterProxy` — мост: Tomcat видит обычный фильтр, а работу делает бин из контекста. В цепочке около 15 фильтров, их порядок фиксирован.

Аутентификация изнутри



401: `AuthenticationException` — кто делает запрос, неизвестно.
403: `AccessDeniedException` — пользователь известен, но прав не хватает

Конфигурация Spring Security

```
@Configuration @EnableMethodSecurity
public class SecurityConfig {
  @Bean
  SecurityFilterChain chain(HttpSecurity http)
      throws Exception {
    return http
        .authorizeHttpRequests(a -> a
            .requestMatchers("/public/**").permitAll()
            .requestMatchers(HttpMethod.GET, "/students/**")
                .authenticated()
            .anyRequest().hasRole("ADMIN"))
        .httpBasic(Customizer.withDefaults())
        .build();
  }
  @Bean PasswordEncoder encoder() {
    return new BCryptPasswordEncoder();
  }
}
@Service public class StudentService {
  @PreAuthorize("hasRole('ADMIN')")   // proxy
  public void delete(long id) { ... }
}
```

- Правила для URL — в бине `SecurityFilterChain`
- `@PreAuthorize` — защита метода через **прокси**: те же ловушки, что у `@Transactional`
- Пароли хранятся только как хеш: `BCryptPasswordEncoder` сам добавляет соль
- Старый базовый класс-адаптер из примеров в интернете удалён в Spring Security 6

Jakarta Security

```
@BasicAuthenticationMechanismDefinition(
    realmName = "students")
@DatabaseIdentityStoreDefinition(
    dataSourceLookup = "java:app/jdbc/StudentsDS",
    callerQuery = "select hash from users where name = ?",
    groupsQuery = "select role from roles where name = ?")
@ApplicationScoped
public class SecurityConfig {}

@Path("/students") @RequestScoped
public class StudentResource {
  @Inject SecurityContext security;

  @DELETE @Path("/{id}") @RolesAllowed("ADMIN")
  public void delete(@PathParam("id") long id) { ... }
}
```

Стандарт с Java EE 8, поверх Jakarta Authentication.

- **Как** получить учётные данные: `HttpAuthenticationMechanism` — Basic, Form, OpenID Connect, свой
- **Где** проверить: `IdentityStore` — БД, LDAP, свой
- Текущий пользователь: `@Inject SecurityContext`
- `@RolesAllowed` проверяет контейнер
- Пароли в БД по умолчанию — `Pbkdf2PasswordHash`

Spring Security и Jakarta Security

	Spring Security	Jakarta Security
Где встроено	цепочка servlet-фильтров	в контейнер сервера
Учётные данные	фильтры: Basic, Form, Bearer	`HttpAuthenticationMechanism`
Проверка пользователя	`UserDetailsService` + `AuthenticationProvider`	`IdentityStore`
Пароли	`PasswordEncoder` (BCrypt)	`Pbkdf2PasswordHash`
Текущий пользователь	`SecurityContextHolder` (`ThreadLocal`)	`@Inject SecurityContext`
Защита метода	`@PreAuthorize` — прокси	`@RolesAllowed` — перехватчик контейнера
Токены	OAuth2 Resource Server	MicroProfile JWT, OpenID Connect

Итоги

Механизм	Spring	Jakarta EE
Основа	рефлексия + аннотации	рефлексия + аннотации
Рецепты	`BeanDefinition` + BFPP	`Bean<T>` + extensions
Обёртки	BPP → прокси	перехватчики, декораторы
Потоки	на разработчике	пул EJB, `@Lock`
Запуск	fat jar	сервер приложений
Вход HTTP	`DispatcherServlet`	Jakarta REST
Безопасность	фильтры Spring Security	контейнер + Jakarta Security
Реализации	Tomcat, Hibernate	те же или другие на выбор

Spring даёт залезть в контейнер руками, Jakarta EE — стандарт и гарантии

Вопросы для самопроверки

1. Почему аннотация без `@Retention(RUNTIME)` бесполезна?
2. Чем Java-аннотация отличается от Python-декоратора?
3. Почему в конструкторе `repeat == 0`?
4. Почему `@Transactional` не работает в `@PostConstruct`?
5. Чем BFPP отличается от BPP и что их заменяет в CDI?
6. Почему в CDI нет проблемы скринсейвера?
7. Как `@Stateless` EJB защищает от гонок?
8. Какие реализации Jakarta EE лежат внутри Spring Boot?
9. Чем servlet-фильтр отличается от `HandlerInterceptor`?
10. Кто и когда возвращает 401, а кто 403?
11. Чем запуск на сервере приложений отличается от fat jar?

Источники

Видео

Е. Борисов, «Spring-потрошитель»
часть 1:
`lektorium.tv/lecture/23982`
часть 2:
`lektorium.tv/lecture/23983`

Документация

Jakarta EE Tutorial
Spring Framework Reference
Spring Security Reference
Spring Boot Reference
Jakarta Security Specification
MicroProfile JWT Specification

Спасибо
за внимание!

ITMO more than a UNIVERSITY